BTS CIEL · PROGRAMMATION C++ · EXERCICES

Exercices — Chapitre 2 : Types, variables, entrées-sorties et opérateurs

Cours associé : `C02_Types_variables_entrees_sorties` · **Fichiers de départ :** `02_Exercices/fichiers/E02/`

> **Méthode :** créez un dossier `E02/` et un fichier par exercice. Compilez avec `g++ -std=c++17 -Wall -Wextra` : un exercice n'est terminé que s'il compile **sans erreur ni avertissement** et que **tous les tests** donnent le résultat attendu.

N°	Titre	Niveau	Notions du cours
2.1	Fiche d'un capteur	1 — Application	§1, §2, §3
2.2	Prévoir l'affichage	1 — Application	§6
2.3	Moyenne de trois mesures	2 — Entraînement	§5, §6.1
2.4	Badge d'accès	2 — Entraînement	§4, §5.3
2.5	Conversion de durée	3 — Problème	§5, §6
2.6	Durée d'un transfert réseau	4 — Défi	§1 à §6

Niveau 1 — Application directe

Exercice 2.1 — Fiche d'un capteur

Objectif : déclarer et initialiser des variables de types adaptés, utiliser une constante, mettre en forme un affichage.

Énoncé : un capteur de température est décrit par :

- son identifiant : `T1` ;
- sa dernière mesure : 21,5 °C ;
- son état : actif ;
- le nombre de mesures déjà enregistrées : 120 ;
- sa période de mesure, **fixe** : 10 secondes.

1. Déclarez une variable de type adapté pour chaque donnée ; la période est une constante.
2. Affichez la fiche du capteur exactement comme ci-dessous.
3. Ajoutez une ligne qui affiche la durée couverte par les mesures enregistrées, en secondes, calculée à partir des variables.

Contraintes : toutes les variables sont initialisées avec `{}` ; noms en `camelCase`, constante en majuscules ; l'état s'affiche `true` ou `false` ; chaque libellé est suivi d'une tabulation (`\t`).

Résultat attendu :

```
Capteur :   T1
Mesure :    21.5
Actif : true
Mesures :   120
Periode :   10 s
Duree : 1200 s
```

Exercice 2.2 — Prévoir l'affichage

Objectif : distinguer division entière, reste et division réelle.

Énoncé : sans exécuter le programme `ex2_2.cpp` ci-dessous, indiquez ce qu'il affiche. Expliquez ensuite pourquoi les deux dernières lignes diffèrent. Vérifiez enfin en compilant.

```cpp
#include <iostream>

int main() {
    int a{9};
    int b{4};
    double c{static_cast<double>(a) / b};

    std::cout << a / b << '\n';
    std::cout << a % b << '\n';
    std::cout << c << '\n';
    std::cout << static_cast<double>(a / b) << '\n';

    return 0;
}
```

Résultat attendu : les quatre lignes affichées, prévues à la main, et une explication en une ou deux phrases.

Niveau 2 — Entraînement

Exercice 2.3 — Moyenne de trois mesures

Objectif : lire des entiers au clavier et calculer une moyenne réelle.

Énoncé : demandez trois mesures entières à l'utilisateur, puis affichez leur moyenne **réelle**.

Contraintes : les trois mesures sont de type `int` ; la moyenne est de type `double`.

Tests :

Saisies	Affichage attendu
`10 12 15`	`Moyenne : 12.3333`
`1 2 2`	`Moyenne : 1.66667`
`4 4 4`	`Moyenne : 4`

Exercice 2.4 — Badge d'accès

Objectif : enchaîner la lecture d'un nombre et d'une ligne complète ; manipuler une `std::string`.

Énoncé : un poste d'accueil enregistre les badges d'accès.

1. Demandez le numéro du badge (entier), puis le nom complet du titulaire (prénom et nom, séparés par un espace).
2. Affichez une ligne récapitulative, l'initiale du titulaire et le nombre de caractères de son nom complet.

Contraintes : le nom complet est lu en une seule fois, espaces compris ; textes saisis sans accents.

Tests :

```
Numero du badge : 1042
Nom complet : Charles Carnus
Badge 1042 : Charles Carnus
Initiale : C
Longueur du nom : 14
```

> **Conseil :** si le programme « saute » la saisie du nom, relisez le §5.3 du cours.

Niveau 3 — Résolution de problèmes

Exercice 2.5 — Conversion de durée

Objectif : utiliser la division entière et le reste pour découper une valeur.

Énoncé : un chronomètre d'équipement donne une durée de fonctionnement en secondes. Demandez cette durée, puis affichez-la en heures, minutes et secondes, en utilisant `/` et `%`.

Contraintes : uniquement des entiers ; aucune boucle, aucune condition.

Tests :

Saisie	Affichage attendu
`3675`	`1 heure(s), 1 minute(s), 15 seconde(s)`
`59`	`0 heure(s), 0 minute(s), 59 seconde(s)`
`3600`	`1 heure(s), 0 minute(s), 0 seconde(s)`
`86399`	`23 heure(s), 59 minute(s), 59 seconde(s)`

Exemple d'exécution :

```
Duree en secondes : 3675
1 heure(s), 1 minute(s), 15 seconde(s)
```

Niveau 4 — Défi

Exercice 2.6 — Durée d'un transfert réseau

Objectif : enchaîner des conversions d'unités sans erreur de division.

Contexte : un technicien doit estimer la durée de copie d'une image système vers un poste, à travers le réseau.

Énoncé :

1. Demandez la taille du fichier en mégaoctets (Mo) et le débit du réseau en mégabits par seconde (Mbit/s). Ces deux valeurs peuvent être décimales.
2. Calculez la durée du transfert en secondes. Un octet vaut 8 bits ; on prend 1 Mo = 1 000 000 octets et 1 Mbit/s = 1 000 000 bit/s.
3. Affichez cette durée, puis la même durée en minutes et secondes entières.

Contraintes :

- constante `constexpr` pour le nombre de bits par octet ;
- pour garder la partie entière d'un `double`, convertissez-le en `int` avec `static_cast<int>(…)` : comme au §1.1 du cours, la partie décimale est supprimée ;
- on ne traite pas les saisies invalides dans cet exercice.

Tests :

Taille (Mo)	Débit (Mbit/s)	Affichage attendu
`700`	`20`	`Duree : 280 s, soit 4 min 40 s`
`4500`	`100`	`Duree : 360 s, soit 6 min 0 s`
`1`	`8`	`Duree : 1 s, soit 0 min 1 s`
`50`	`3`	`Duree : 133.333 s, soit 2 min 13 s`

Pour aller plus loin : le débit réel d'un réseau est inférieur au débit nominal (protocoles, partage du lien). Ajoutez une constante d'efficacité (par exemple 0,8) et affichez aussi la durée réaliste.